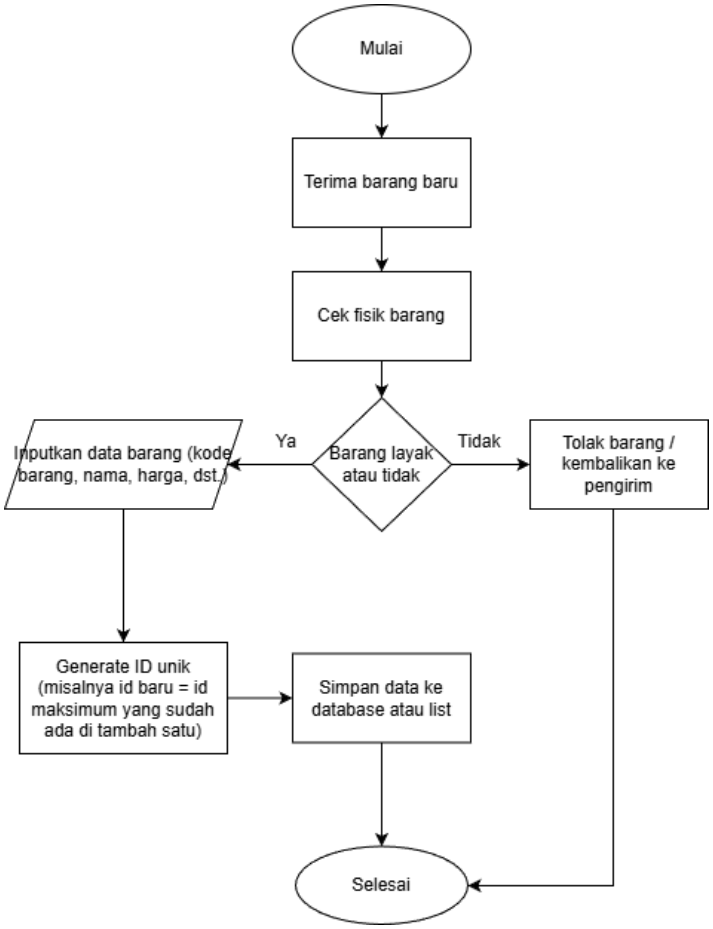
Nama : Muhammad Aziz Syah Dani
NIM : F1E325063
PROJEK ALGORITMA PEMROGRAMAN (UAS)

Studi Kasus: WareHouse OS (Sistem Manajemen Gudang)

Modul A: Alur Penerimaan Barang (Pengantar & Logika Dasar)
Konteks: Gudang menerima barang baru setiap hari. Barang harus dicek kondisinya, dicatat, dan diberikan label.

Tugas Mahasiswa:

1. Membuat **Flowchart** dari proses barang masuk (Mulai → Cek Fisik → Input Data → Generate ID → Simpan → Selesai).



2. Menulis **Pseudocode** dan deskripsi algoritma untuk fungsi input_barang().

```
FUNCTION input_barang(data)

TULIS "[Input Barang Baru]"

TULIS "Masukkan kondisi fisik barang (Baik atau Rusak)"
```

```
BACA kondisi

JIKA kondisi bukan "Baik" dan bukan "baik" DAN bukan
"bagus"
     TULIS "Barang ditolak karena kondisi tidak baik"
     KEMBALIKAN

TULIS "Masukkan kode barang"
BACA kode

TULIS "Masukkan nama barang"
BACA nama

TULIS "Masukkan harga"
BACA harga

TULIS "Masukkan tahun kedaluwarsa"
BACA tahun_kedaluwarsa

TULIS "Masukkan nomor rak"
BACA nomor_rak

TULIS "Masukkan tanggal masuk (format YYYY-MM-DD)"
BACA tanggal_masuk

TULIS "Masukkan jumlah"
BACA jumlah

JIKA data tidak kosong
     new_id ← nilai maksimum id_resi di dalam data + 1
JIKA data kosong
     new_id ← 1

BUAT objek barang_baru dari struktur Barang:
     id_resi ← new_id
     kode_barang ← kode
     nama_barang ← nama
     harga ← harga
     tahun_kedaluwarsa ← tahun_kedaluwarsa
     kondisi_fisik ← kondisi
     nomor_rak ← nomor_rak
     tanggal_masuk ← tanggal_masuk
     jumlah ← jumlah
```

```
TAMBAHKAN barang_baru ke dalam list data
TULIS "Barang berhasil disimpan dengan ID resi = ",
new_id

END FUNCTION
```

3. **Implementasi:** Menggunakan paradigma prosedural untuk struktur data dasar (struct/class).

   Hasil implementasi ada di dalam file modul_a.py

Modul B: Analisis Efisiensi (Kompleksitas Waktu & Ruang)

Konteks: Gudang memiliki kapasitas memori terbatas dan waktu pemrosesan harus cepat saat data mencapai 10.000 item.

Tugas Mahasiswa:

1. Membandingkan dua algoritma yang akan digunakan nanti (misal: Bubble Sort vs Quicksort).

   Bubble Sort itu membandingkan elemen-elemen berurutan satu per satu dan menukar bila perlu

➔ Setiap putaran akan menempatkan elemen paling besar di posisi akhir
➔ Untuk n barang perlu banyak putaran sehingga lambat jika n besar

Quick Sort memilih satu elemen sebagai pivot, membagi data menjadi tiga bagian kurang dari pivot, sama dengan pivot, lebih dari pivot

➔ Proses tersebut dipanggil berulang secara rekursi untuk bagian kiri dan kanan
➔ Jika pembagian cukup seimbang maka jumlah langkah menjadi jauh lebih sedikit

2. Menghitung **Big-O Notation** (O(n), O(n^2), dll) untuk skenario terburuk (*worst case*) dan terbaik (*best case*).

   Bubble Sort

Kondisi	**Kompleksitas**	**Penjelasan**
Worst Case	O(n^2)	Data terbalik
Best Case	O(n)	Data sudah terurut, hanya satu kali lewat, kompleksitas waktu
Average Case	O(n^2)	Tetap melakukan perbandingan sekitar n^2

➔ Kompleksitas ruang: O(1) karena hanya memakai beberapa variabel tambahan
Quick Sort

Kondisi	**Kompleksitas**	**Penjelasan**
Worst Case	O(n^2)	Pivot selalu menajdi elemen paling kecil atau paling besar sehingga pembagian sangat tidak seimbang
Best Case	O(n log n)	Data selalu terbagi cukup seimbang
Average Case	O(n log n)	Sama seperti best case

➔ Kompleksitas ruang: O(log n) untuk tumpukan rekursi

Modul C: Pengurutan Inventaris (Algoritma Sorting)

Konteks: Staff gudang perlu melihat daftar barang dalam berbagai urutan untuk laporan harian.

Tugas Mahasiswa:

1. **Bubble Sort:** Fitur untuk mengurutkan barang berdasarkan *Harga Termurah* (Hanya untuk pembelajaran konsep dasar & perbandingan performa lambat).

```python
def bubblesort(data: list[Barang]) -> list[Barang]:
    arr = data[:]
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(0, n - 1 - i):
            if arr[j].harga > arr[j + 1].harga:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

   Bubble Sort dipakai khusus untuk belajar konsep dasar dan menunjukkan contoh algoritma sederhana yang kurang efisien. Pada fungsi `bubblesort`, data barang disalin lalu diurutkan berdasarkan field harga. Setiap iterasi luar melakukan beberapa perbandingan di bagian dalam. Jika tidak ada pertukaran dalam satu putaran, algoritma berhenti karena data sudah terurut. Walaupun lambat untuk data besar, implementasi ini sangat jelas dan mudah diikuti, sehingga cocok untuk menjelaskan urutan langkah ke dosen.

2. **Selection Sort:** Fitur wajib untuk mengurutkan barang berdasarkan *Tahun Kedaluwarsa (Expired Date)*. Mahasiswa harus menjelaskan kenapa algoritma ini dipilih (meminimalkan jumlah pertukaran/swapping).

```python
def selectionsort(data: list[Barang]) -> list[Barang]:
    arr = data[:]
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j].tahun_kedaluwarsa < arr[min_index].tahun_kedaluwarsa:
                min_index = j
        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
```

   Algoritma Selection Sort digunakan untuk menyusun barang berdasarkan tahun kedaluwarsa. Pertimbangannya adalah selection sort meminimalkan jumlah pertukaran. Dalam setiap iterasi luar, algoritma hanya mencari indeks elemen dengan tahun kedaluwarsa paling kecil di bagian yang belum terurut.

Setelah itu dilakukan satu kali pertukaran dengan posisi awal. Jumlah swap yang sedikit ini sejalan dengan ide mengurangi biaya pemindahan data. Di konteks gudang, hal tersebut dapat dianalogikan sebagai upaya meminimalkan perpindahan fisik atau biaya update data yang besar.

3. **Quicksort & Recursion:** Fitur untuk mengurutkan barang berdasarkan *ID Resi (Nomor Unik)* dari terkecil ke terbesar. Ini wajib menggunakan konsep **Rekursi**.

```python
def quicksort(data: list[Barang]) -> list[Barang]:
    if len(data) <= 1:
        return data[:]
    pivot = data[len(data) // 2].id_resi
    left = [b for b in data if b.id_resi < pivot]
    middle = [b for b in data if b.id_resi == pivot]
    right = [b for b in data if b.id_resi > pivot]
    return quicksort(left) + middle + quicksort(right)
```

Quick Sort dipakai untuk mengurutkan barang berdasarkan id resi. Fungsi `quicksort` menggunakan rekursi secara eksplisit. Jika panjang data satu atau nol, data langsung dikembalikan. Bila lebih dari itu, algoritma memilih pivot berupa id resi di posisi tengah, lalu membentuk tiga list baru yaitu bagian kiri, tengah, dan kanan. Fungsi yang sama dipanggil lagi untuk bagian kiri dan kanan, kemudian hasilnya digabung. Bentuk ini jelas menggambarkan prinsip divide and conquer dan penggunaan rekursi, sehingga memenuhi syarat rubrik bahwa Quicksort harus didemonstrasikan dengan rekursi.

Buat implementasi dari semua point di atas ada di file ***`modul_c.py`***
Sesuai tugas nya dia akan membuat file baru bernama ***`data_barang_c.csv`*** *dikarenakan dibutuhkan untuk Modul D untuk membaca data barang hasil output dari Implementasi Modul C*

Modul D: Pencarian Barang (Algoritma Searching)

Konteks: Kurir datang untuk mengambil paket tertentu. Admin harus menemukan barang tersebut secepat mungkin.

Tugas Mahasiswa:

1. **Linear Search:** Fitur cari barang berdasarkan *Nama Barang* (karena nama barang bisa tidak urut/acak).

```python
def linearsearch(data: list[Barang], keyword: str) -> list[Barang]:
    hasil: list[Barang] = []
    key = keyword.lower()
    for b in data:
        if key in b.nama_barang.lower():
            hasil.append(b)
    return hasil
```

Pencarian berdasarkan nama barang menggunakan Linear Search. Nama barang pada gudang bersifat teks bebas dan tidak selalu tersusun dalam urutan tertentu, bahkan bisa memiliki kata yang mirip. Fungsi `linearsearch` menelusuri seluruh list barang satu per satu. Setiap nama barang diubah ke huruf kecil, kemudian dicek apakah mengandung kata kunci yang dimasukkan pengguna. Semua barang yang cocok dikumpulkan dalam list hasil dan dikembalikan. Karena mengecek satu per satu, kompleksitas waktunya O(n). Meskipun demikian, cara ini fleksibel dan cocok untuk situasi di mana urutan data tidak terjamin.

2. **Binary Search:** Fitur cari barang berdasarkan *ID Resi*. (Syarat: Data harus di-sort terlebih dahulu menggunakan Quicksort dari Modul C).

```python
def binarysearch(sorted_data: list[Barang], target: int) -> Barang | None:
    low = 0
    high = len(sorted_data) - 1
    while low <= high:
        mid = (low + high) // 2
        mid_id = sorted_data[mid].id_resi
        if mid_id == target:
            return sorted_data[mid]
        elif mid_id < target:
            low = mid + 1
        else:
            high = mid - 1
    return None
```

Pencarian berdasarkan id resi menggunakan Binary Search. Syarat pentingnya adalah data harus sudah terurut terlebih dahulu berdasarkan id resi. Setelah data terurut, fungsi `binarysearch` melakukan pencarian dengan cara membagi dua area pada setiap langkah. Nilai id resi di tengah dibandingkan dengan target. Jika sama maka barang ditemukan. Jika kurang dari target, pencarian dilanjutkan di bagian kanan. Jika lebih besar, pencarian dilanjutkan di bagian kiri. Proses berhenti ketika rentang indeks habis atau barang ditemukan. Secara teori, kompleksitas waktu pencarian ini adalah O(log n), jauh lebih cepat daripada linear search untuk data besar.

file full hasil implementasi nya ada di modul_d.py

Modul E: Penyimpanan Cepat (Hash Tables)

Konteks: Gudang memiliki ribuan rak. Mencari lokasi rak satu per satu (looping) terlalu lama.

Tugas Mahasiswa:

1. Membuat sistem **Hash Table** untuk memetakan Kode_Barang ke Nomor_Rak.

```python
class HashTableLokasiRak:
    def __init__(self, size: int = 101):
        self.size = size
        self.buckets = [[] for _ in range(size)]

    def _hash(self, kode_barang: str) -> int:
        total = 0
        for ch in kode_barang:
            total += ord(ch)
        return total % self.size

    def insert(self, kode_barang: str, nomor_rak: int) -> None:
        index = self._hash(kode_barang)
        bucket = self.buckets[index]

        for i, (k, v) in enumerate(bucket):
            if k == kode_barang:
                bucket[i] = (kode_barang, nomor_rak)
                return

        bucket.append((kode_barang, nomor_rak))

    def get(self, kode_barang: str):
        index = self._hash(kode_barang)
        bucket = self.buckets[index]
        for k, v in bucket:
            if k == kode_barang:
                return v
        return None
```

Kelas `HashTableLokasiRak` memiliki sebuah array bucket dengan ukuran tertentu. Fungsi `_hash` mengubah kode barang yang berupa string menjadi sebuah indeks dengan cara menjumlahkan nilai karakter lalu mengambil sisa bagi terhadap ukuran tabel. Pendekatan ini cukup sederhana tetapi sudah cukup untuk mendistribusikan kode barang ke berbagai bucket. Saat fungsi `insert` dipanggil, struktur akan mencari bucket berdasarkan hasil hash. Jika kode barang tersebut sudah ada, nomor rak diperbarui. Jika belum ada, pasangan kode barang dan nomor rak ditambahkan ke bucket itu. Metode ini disebut separate chaining karena satu indeks dapat menyimpan beberapa entri dalam bentuk list kecil.

2. Tujuannya agar pencarian lokasi rak memiliki kompleksitas waktu mendekati $O(1)$ (instan), tanpa perlu searching berulang.

```python
def build_hash_table(data: list[Barang]) -> HashTableLokasiRak:
    ht = HashTableLokasiRak()
    for b in data:
        ht.insert(b.kode_barang, b.nomor_rak)
    return ht
```

Fungsi **build_hash_table** membangun Hash Table dari seluruh data barang yang sudah dibaca dari file csv. **modul_e.py** kemudian memberikan antarmuka sederhana. Pengguna cukup memasukkan kode barang, dan program akan menampilkan nomor rak yang terkait. Dengan demikian implementasi memenuhi rubrik untuk struktur data canggih yaitu penggunaan Hash Table untuk lookup mendekati O(1) dan integrasi dengan data simulasi gudang.

Algoritma ini menyusun data berdasarkan harga, disusun dari nilai terkecil ke tersbesar. Setelah itu algoritma ini membuat file baru yang berisi data yang telah disusun/disortir ke dalam file baru, berikut adalah perbandingan data sebelum dan sesudah sortir.

hasil implementasi dari kode tugas tersebut ada di file modul_e.py dan setelah kamu menjalankan file modul_e.py maka kamu akan mendapat kan file baru bernama data_peta.csv dikarenakan hasil output dari data barang yang sudah di petakan oleh program tersebut akan masuk ke dalam data_peta.csv